

Assignment 2B Classification Metrics LLM Transcript –

Brandon Chanderban

The following outlines the interactions I had with the LLM during the completion of this assignment, arranged in the order they occurred. Questions are denoted in green.

Q: Can you assist in generating a ggplot visualization of the class distribution of the sex variable within the penguin dataframe? Have them be differentiated by color and include a label of the count at the end of the bars.

Certainly! Here's how you could generate a **ggplot2** bar chart that visualizes the distribution of the sex variable in your penguin dataset, with:

- Differentiated colors for each class (male, female)
- Count labels placed above each bar

✅ LLM Response:

Yes! Here's the R code using **ggplot2** to create a colored bar plot showing the class distribution of the sex variable:

```
library(ggplot2)
```

```
ggplot(df, aes(x = sex, fill = sex)) +  
  geom_bar() +  
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.5) +  
  scale_fill_manual(values = c("male" = "steelblue", "female" = "salmon")) +  
  labs(  
    title = "Distribution of Actual Class Labels (Sex)",  
    x = "Sex",  
    y = "Count"  
  ) +  
  theme_minimal()
```

🔍 Explanation:

- `aes(x = sex, fill = sex)`: Plots the sex variable on the x-axis and fills the bars based on the category (male or female).
- `geom_bar()`: Creates the bar chart showing counts of each class.
- `geom_text(...)`: Adds the count above each bar using `after_stat(count)` for label text.
- `scale_fill_manual(...)`: Assigns custom colors to each class.
- `theme_minimal()`: Applies a clean, minimalistic theme for better aesthetics.

This plot will help you visually assess class imbalance in the dataset — a crucial step when evaluating binary classification models.

Q: How would I go about calculating the null error rate of the penguin dataframe within R? Could you both explain the solution code and what the null error rate communicates.

Absolutely. Here is a clear, assignment-appropriate explanation that matches the level and wording style you've been using, and is suitable for inclusion in an **LLM transcript**.

LLM Response

To calculate the **null error rate** of the penguin dataset in R, we first need to identify the **majority class** in the target variable (sex). The null model is a baseline classifier that always predicts the most common class, regardless of input features.

R Code

```
library(dplyr)

# Count observations in each class
class_counts <- df %>% count(sex)

# Total number of observations
total_obs <- nrow(df)

# Number of observations in the majority class
majority_class_n <- max(class_counts$n)

# Null error rate
null_error_rate <- 1 - (majority_class_n / total_obs)

# Display the result
cat("Null Error Rate:", round(null_error_rate, 3))
```

Step-by-Step Explanation

- `count(sex)` calculates how many observations belong to each class (male and female).
- `nrow(df)` stores the total number of observations in the dataset.
- `max(class_counts$n)` identifies the size of the majority class.
- The null error rate is computed as:

$$\text{Null Error Rate} = 1 - \frac{\text{Majority Class Count}}{\text{Total Observations}}$$

This represents the proportion of incorrect predictions that would occur if we **always predicted the majority class**.

What the Null Error Rate Communicates

The null error rate provides a **baseline level of performance** against which more sophisticated models can be compared. If a classification model does not outperform

the null error rate, then it offers no practical improvement over a trivial guessing strategy.

Additionally:

- A **high null error rate** typically indicates a more balanced dataset and a more challenging classification task.
- A **low null error rate** suggests class imbalance, where a model may appear accurate simply by predicting the dominant class—potentially masking poor performance on the minority class.

For these reasons, the null error rate is a critical reference point when evaluating whether a classification model is meaningfully effective.

Q: Given that I already created three additional columns for three different thresholds (pred_02 for 0.2, pred_05 for 0.5, and pred_08 for 0.8), how can I use R to obtain confusion counts and therein create three confusion matrices within a singular table?

LLM Response

Since you have already created predicted class columns for each threshold (pred_02, pred_05, and pred_08), you can compute the confusion matrix components (TP, FP, TN, FN) for each threshold by comparing each predicted column against the actual class column.

A convenient and reusable way to do this is to define a small helper function that calculates the four confusion counts, and then apply it to each threshold.

R Code

```
library(dplyr)
library(tibble)

# Helper function to compute confusion matrix counts
get_confusion_counts <- function(predicted, actual) {
  tibble(
    TP = sum(predicted == 1 & actual == 1),
    FP = sum(predicted == 1 & actual == 0),
    TN = sum(predicted == 0 & actual == 0),
    FN = sum(predicted == 0 & actual == 1)
  )
}

# Compute confusion counts for each threshold
cm_02 <- get_confusion_counts(df$pred_02, df$actual)
```

```
cm_05 <- get_confusion_counts(df$pred_05, df$actual)
cm_08 <- get_confusion_counts(df$pred_08, df$actual)
```

```
# Combine results into a single table
```

```
confusion_matrices <- bind_rows(
  `Threshold: 0.2` = cm_02,
  `Threshold: 0.5` = cm_05,
  `Threshold: 0.8` = cm_08,
  .id = "Threshold"
)
```

```
confusion_matrices
```

Explanation

1. Helper function (get_confusion_counts)

- This function takes two vectors:
 - predicted: a binary prediction vector (0 or 1)
 - actual: the true class labels (0 or 1)
- It computes:
 - **True Positives (TP)**: predicted 1 and actual 1
 - **False Positives (FP)**: predicted 1 but actual 0
 - **True Negatives (TN)**: predicted 0 and actual 0
 - **False Negatives (FN)**: predicted 0 but actual 1
- The result is returned as a tidy tibble.

2. Applying the function to each threshold

- The function is applied separately to pred_02, pred_05, and pred_08.
- Each call produces a one-row confusion matrix summary.

3. Combining the confusion matrices

- bind_rows() stacks the three results into a single table.
 - The .id = "Threshold" argument creates a new column identifying which threshold each row corresponds to.
-

Outcome

The resulting table presents **three confusion matrices in a single, tidy format**, allowing you to directly compare how the number of true positives, false positives, true negatives, and false negatives changes as the probability threshold increases.

Q: How can I then compute performance metrics, using the recently generated confusion matrices, and have that information be displayed in a well-spaced and visualized chart?

Great question! Once you've computed your **confusion matrices** (with TP, FP, TN, and FN), you can calculate performance metrics — **Accuracy, Precision, Recall, and F1 Score** — using these values. Then, you can display the results in a clean, well-formatted table using the kableExtra package.

✅ Step-by-Step Solution

1. Calculate performance metrics

Use mutate() on your confusion matrix table to calculate each metric per threshold:

```
library(dplyr)
```

```
metrics <- confusion_matrices %>%  
  mutate(  
    Accuracy = (TP + TN) / (TP + FP + TN + FN),  
    Precision = TP / (TP + FP),  
    Recall = TP / (TP + FN),  
    F1 = 2 * (Precision * Recall) / (Precision + Recall)  
  ) %>%  
  select(Threshold, TP, FP, TN, FN, Accuracy, Precision, Recall, F1)
```

2. Format and display as a nice chart

You can use kableExtra to make a well-styled, spaced, and readable table:

```
library(knitr)
```

```
library(kableExtra)
```

```
kable(metrics, digits = 3, caption = "Performance Metrics at Different Thresholds") %>%  
  kable_styling(  
    bootstrap_options = c("striped", "hover", "condensed", "responsive"),  
    full_width = FALSE,  
    position = "center"  
  ) %>%  
  row_spec(0, bold = TRUE, background = "#f5f5f5")
```

🧠 Explanation of Each Metric

Metric	What it Tells You
--------	-------------------

Accuracy	Overall how many predictions were correct
-----------------	---

Precision	Of all predicted positives, how many were actually correct
------------------	--

Recall	Of all actual positives, how many were correctly predicted
---------------	--

F1 Score	Harmonic mean of precision and recall (balances both)
-----------------	---

Output Format

This will produce a **clean HTML/Quarto table** in your document that shows, per threshold:

- The raw confusion matrix values (TP, FP, TN, FN)
- The computed metrics (Accuracy, Precision, Recall, F1)

You'll be able to easily compare how performance changes across different threshold levels.

Let me know if you'd also like a bar plot or line plot of these metrics!